

SUMMARY

Regular Expressions (RE) function as the formal representers of Regular Languages in the Theory of Computation.

Finite Automata act as acceptors.

Regular Grammars act as generators.

Regular Expressions act as representers.

Every regular language can be represented by at least one regular expression. However, the representation is not unique.

Mastery of unary and binary operators, closure properties, algebraic identities, and simplification rules is essential for solving competitive examination problems efficiently.

1. THEORETICAL FRAMEWORK OF REGULAR LANGUAGES

Regular languages are studied from three complementary perspectives.

Perspective	Tool	Role
Acceptance	Finite Automata (DFA, NFA, ϵ -NFA)	Accept strings
Generation	Regular Grammar (LLG, RLG)	Generate strings
Representation	Regular Expression	Represent string patterns

Core Principles:

Representer:

A Regular Expression represents a Regular Language.

Exclusivity:

Regular Expressions exist only for Regular Languages.

They cannot represent Context-Free or Recursively Enumerable languages.

Existence:

If a language is regular, a corresponding regular expression always exists.

2. FUNDAMENTAL NOTATIONS AND OPERATORS

Basic Symbols and Meanings

Notation	Meaning	Language Description
$r = \varnothing$	Empty Set	$L = \varnothing$
$r = \varepsilon$	Empty String	$L = \{\varepsilon\}$
$r = a$	Literal	$L = \{a\}$
$a + b$	Union / OR	$L = \{a, b\}$
ab	Concatenation	$L = \{ab\}$
a^*	Kleene Closure	$\{\varepsilon, a, aa, aaa, \dots\}$
a^+	Positive Closure	$\{a, aa, aaa, \dots\}$

Operator Classification

Unary Operators:

- Kleene Closure (*)
- Positive Closure (+)

Binary Operators:

- Concatenation
- Union (+)

Important Note:

The symbol "+" serves dual purposes:

Unary $\rightarrow$ Positive closure

Binary $\rightarrow$ Union

3. ALGEBRAIC IDENTITIES AND SIMPLIFICATIONS

Closure Identities

Identity	Result
$R^* = \epsilon + R^+$	Kleene includes positive plus empty
$R^* R = R^+$	Closure followed by base equals positive
$R^* R^* = R^*$	Repeated closure redundant
$(R^*)^* = R^*$	Nested closure redundant
$(R^+)^+ = R^+$	Nested positive redundant

Null and Identity Properties

Identity	Result
$\varnothing^* = \epsilon$	Closure of empty set
$\epsilon^* = \epsilon$	Closure of empty string
$R + \varnothing = R$	Union with empty set
$R \varnothing = \varnothing$	Concatenation with empty set
$R \epsilon = R$	Concatenation with empty string

Distribution and Commutation

Property	Observation
$R_1 + R_2 = R_2 + R_1$	Union is commutative
$R_1 R_2 \neq R_2 R_1$	Concatenation not commutative

Factoring Example:

$$\begin{aligned}
 &a + ab \\
 &= a(\epsilon + b)
 \end{aligned}$$

Position must be preserved during factoring.

4. ADVANCED PROPERTIES AND SHIFT RELATIONS

Shifting Property

$$R1 (R2 R1)^* = (R1 R2)^* R1$$

This identity allows cyclic shifting of repeating patterns.

Union of Closures (Five-Star Property)

The following expressions are equivalent:

1. $(R1 + R2)^*$
2. $(R1^* + R2^*)^*$
3. $(R1^* R2^*)^*$
4. $(R1^* + R2)^*$
5. $(R1 + R2^*)^*$

All represent closure over the union of base operands.

5. CONSTRUCTION OF SPECIALIZED LANGUAGES

Even and Odd Length (Alphabet $\Sigma = \{a\}$)

Even Length:

$(aa)^*$

Generates strings of length:

0, 2, 4, 6, ...

Odd Length:

$a(aa)^*$

or

$(aa)^*a$

Generates strings of length:

1, 3, 5, 7, ...

Parity Insight:

Even + Even $\rightarrow$ Even

Odd + Even $\rightarrow$ Odd

Counting Strings in Binary Alphabet

For $\Sigma = \{0,1\}$

Total strings up to length n :

$$2^0 + 2^1 + 2^2 + \dots + 2^n$$

Formula:

$$\text{Total Strings} = 2^{(n+1)} - 1$$

Example:

For $n = 5$

$$\text{Total} = 2^6 - 1 = 63$$

6. KEY INSIGHTS FOR PROBLEM SOLVING

Non-Uniqueness:

A regular expression defines exactly one language.

But a single regular language may have multiple equivalent regular expressions.

NFA Parallel:

A regular expression behaves like an NFA — it represents valid patterns but does not explicitly reject invalid ones.

Infinite vs Finite Languages:

R^* and R^+ usually produce infinite languages.

Exceptions:

$$R = \varnothing$$

$$R = \varepsilon$$

These yield finite languages under closure.

Equivalence Testing:

Two expressions R_1 and R_2 are equal if:

$$L(R1) = L(R2)$$

Language equality determines expression equality.

Shortest Missing String Strategy:

1. Test ϵ
2. Test strings of length 1
3. Test strings of length 2
4. Continue incrementally

Stop at first string not generated.

